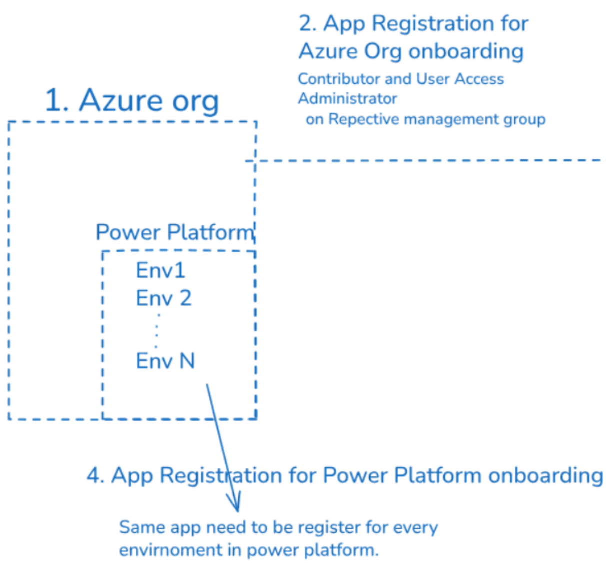
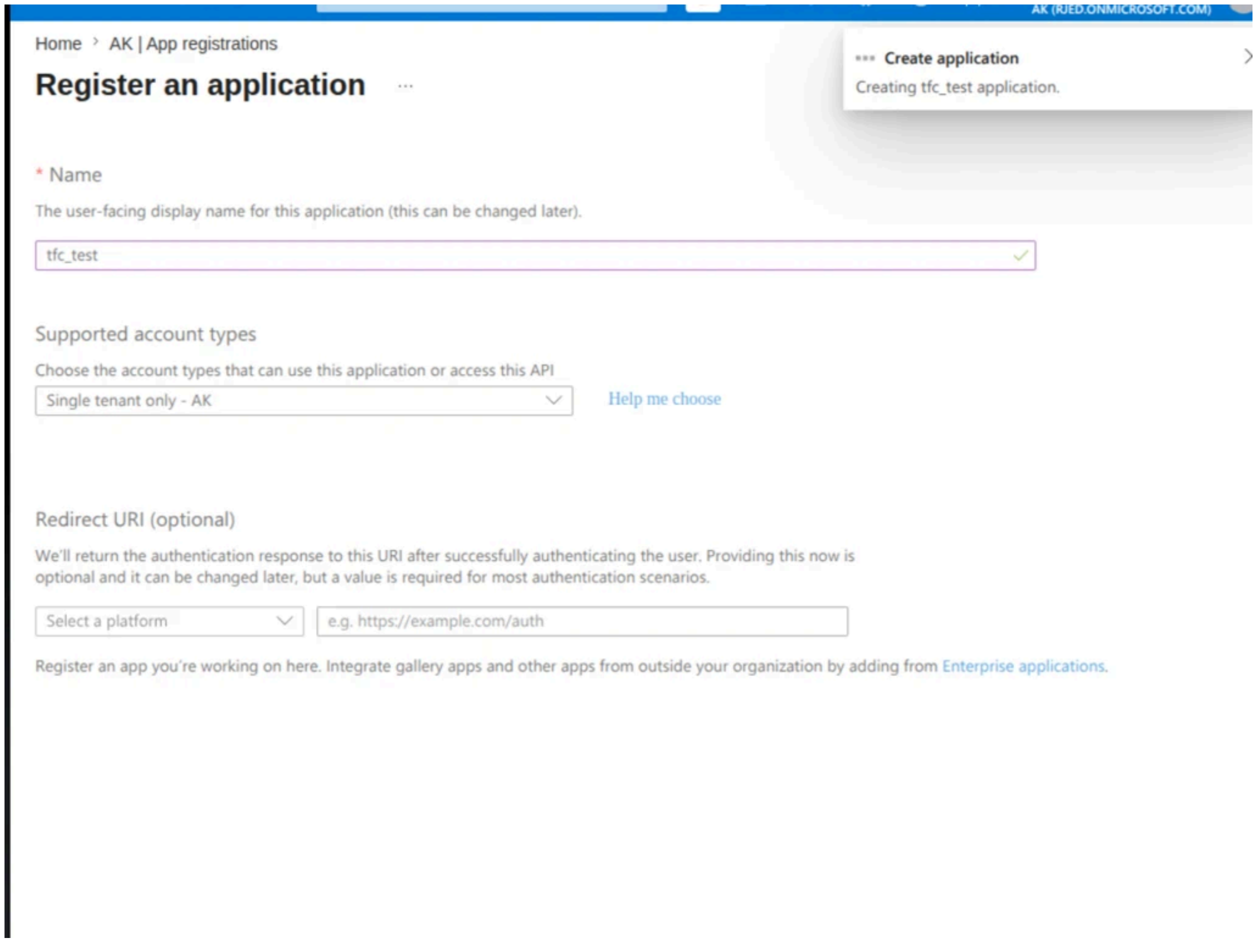
Azure + Power Platform onboarding — setup steps and Terraform run



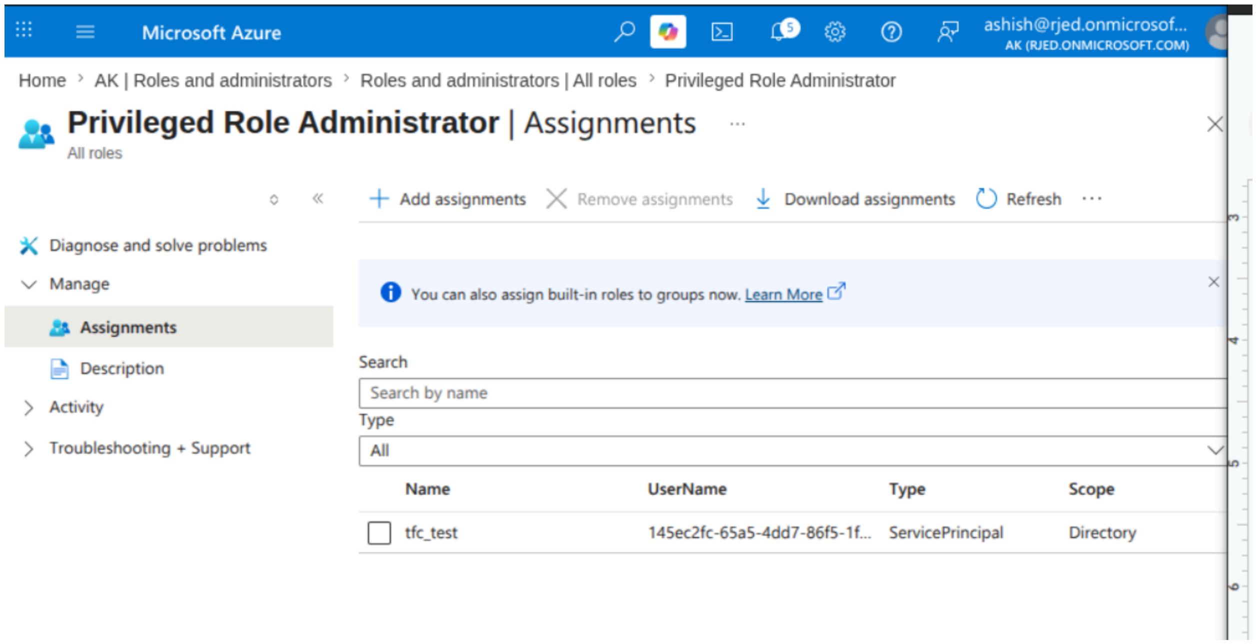
App Registration

- Go to **Microsoft Entra ID** → **App registrations** → **New registration**.
- Create the application name
- Open the **tfc_test** application.
- Go to **Certificates & secrets**.
- Create a **New client secret**.



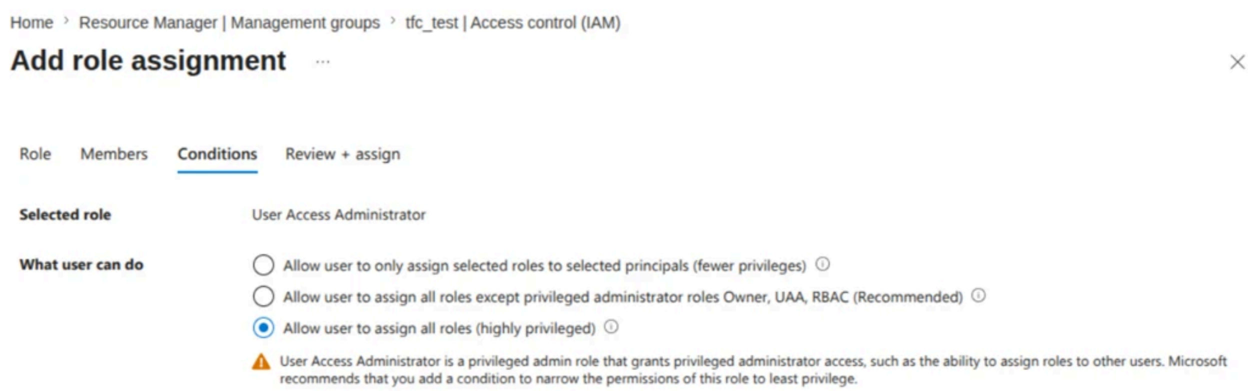
Assign Privileged Role Administrator

- Go to **Microsoft Entra ID → Roles and administrators**.
- Open **Privileged Role Administrator**.
- Click **Add assignments**.
- Select the **registered app** application.
- Assign the role.



Assign Management Group Roles

- Go to **Management Groups**.
- Select the target management group (root).
- Open **Access Control (IAM)**.
- Click **Add role assignment**.
- Assign the following roles to the **registered app**:
  - **Contributor** *(required for resource management)*
  - **User Access Administrator** *(required for role assignments)*



Register the App with Power Platform

- Sign in to Azure using the target tenant.
- Register app as a **Power Platform Management Application**.
- Verify the registration is successful.

Reference:
https://registry.terraform.io/providers/microsoft/power-platform/latest/docs/guides/client_secret

```
az login

CLIENT_ID="<CLIENT_ID>"
TOKEN=$(az account get-access-token \
 --resource "https://api.bap.microsoft.com/" \
 --query accessToken -o tsv)

curl -s -X PUT \
```

```
"https://api.bap.microsoft.com/providers/Microsoft.BusinessAppPlatform/adminApplications/${CLIENT_ID}?api-version=2020-10-01" \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -w "\nHTTP %{http_code}\n"
```

validate the registration

```
curl -s
"https://api.bap.microsoft.com/providers/Microsoft.BusinessAppPlatform/adminApplications?api-version=2020-10-01" \
 -H "Authorization: Bearer $TOKEN" \
| python3 -m json.tool \
| grep -i "$CLIENT_ID" \
&& echo "OK: registered" || echo "NOT registered"
```

Register the App in All Powerplatform Environments

- Register app as an **Application User** in all [Powerplateform environments](#).
- The app is added with the **System Administrator** security role.

  Note: To onboard only specific environments, set the Terraform variable `only_environment_display_names` with the display names of the target environments. If this variable is not specified, the app is registered in all Power Platform environments.

```
#!/usr/bin/env bash
set -euo pipefail

# ── Config ──────────────────────────────────────────────
```

```
TENANT_ID="1117211b-da09-4cdd-b221-67a5e164ece7"
BOOTSTRAP_APP_ID="145ec2fc-65a5-4dd7-86f5-1f0b9ab4a1cf"
API_VERSION="2020-10-01"
# ──────────────────────────────────────────────────────────────────────

BAP="https://api.bap.microsoft.com/providers/Microsoft.BusinessAppPlatfo
rm"

command -v jq >/dev/null || { echo "ERROR: jq is required but not
installed."; exit 1; }

echo ">> Getting BAP admin token..."
TOKEN=$(az account get-access-token --resource
"https://api.bap.microsoft.com/" --query accessToken -o tsv)

echo ">> Listing all environments..."
ENVS=$(curl -sf
"$BAP/scopes/admin/environments?api-version=$API_VERSION" \
 -H "Authorization: Bearer $TOKEN" \
 | jq -r '.value[]
     | select(.name and
.properties.linkedEnvironmentMetadata.instanceUrl)
     | "\(.name)\t\(.properties.linkedEnvironmentMetadata.instanceUrl |
sub("/$";""))"')

COUNT=$(printf '%s\n' "$ENVS" | grep -c . || true)
echo ">> Found $COUNT Dataverse environment(s)."

OK=0; FAIL=0
while IFS=$'\t' read -r ENV_ID ENV_URL; do
 [ -z "$ENV_ID" ] && continue
 echo ""
 echo "=== $ENV_ID  ($ENV_URL) ==="

 RESP=$(curl -s -w $'\n%{http_code}' -X POST \

"$BAP/scopes/admin/environments/${ENV_ID}/addAppUser?api-version=$API_VE
RSION" \
   -H "Authorization: Bearer $TOKEN" \
```

```
   -H "Content-Type: application/json" \
   -d "{\"servicePrincipalAppId\":\"$BOOTSTRAP_APP_ID\"}")

 HTTP=$(printf '%s' "$RESP" | tail -n1)
 BODY=$(printf '%s' "$RESP" | sed '$d')

 case "$HTTP" in
   2*) echo "   OK (HTTP $HTTP) — added as System Administrator";
OK=$((OK+1)) ;;
   *)  echo "   FAILED (HTTP $HTTP)"; echo "$BODY"; FAIL=$((FAIL+1)) ;;
 esac
done <<< "$ENVS"

echo ""
echo ">> Done. Success/skip: $OK   Failed: $FAIL"
[ "$FAIL" -eq 0 ] || { echo "Some environments failed — see output
above."; exit 1; }
```